Risk Profiler Review and Validate Automation

Overview

Risk Profiler Review and Validate Automation is a Chrome Manifest V3 extension for Risk Profiler assessment review, validation, reporting, and documentation. It helps users load Risk Profiler application assessments from Cairo, filter and select assessments, validate selected assessments against a defined checkpoint list, review assessment migration/readiness items, export validation results to Excel, and download formatted Word review notes with selected questions, ASA Notes, contact details, and clickable checkbox controls.

The extension is designed for business users who need consistent review output and for technical teams who need traceable, repeatable, browser-based automation across Cairo, ESATS, and GTC data sources.

Business Purpose

Risk Profiler assessment review requires users to compare completed or incomplete assessments against current survey templates, inspect unanswered reachable questions, validate required checkpoints, collect application context, and prepare notes for follow-up. This extension reduces manual work by automating the repetitive parts of that process.

Business value:

- Standardizes review and validation output across applications.
- Reduces manual lookup across Cairo, ESATS, and GTC.
- Improves repeatability for quality checks and audit support.
- Creates downloadable Excel validation reports.
- Creates Word review notes with application metadata, contacts, dates, selected review output, ASA Notes, and clickable checkbox controls.
- Helps reviewers focus on risk and remediation decisions instead of API navigation and data assembly.

1. Technical Stack

1(a). Full Stack

The extension uses the following stack:

Layer	Technology	Purpose
Browser extension platform	Chrome Extension Manifest V3	Runs the popup UI and service worker automation inside Chrome.
UI	HTML, CSS, vanilla JavaScript ES modules	Popup interface, assessment filters, tabs, progress, results, modals, review notes.

Layer	Technology	Purpose
Background execution	MV3 service worker	Assessment refresh, validation jobs, review jobs, local persistence, scheduled refresh.
Bundling	esbuild	Bundles `popup.js` and `service_worker.js` into `dist/`.
Storage	`chrome.storage.local` with `unlimitedStorage`	Stores assessments, validation results, review results, selected review mode, progress, contexts, ASA Notes, and UI state.
API access	`fetch()` with browser cookies and trusted-tab script execution	Calls Cairo directly and calls ESATS/GTC through signed-in trusted tabs where needed.
Validation engine	Local JavaScript checkpoint modules `RP1` through `RP13`	Runs deterministic validation rules against assembled context, normalized answers, review approvals, and conditional question-summary data.
Review engine	Local JavaScript conversion of the provided Python reachable-unanswered-work-queue algorithm	Compares old/new questions and answers, applies review mode configuration, computes reachable unanswered review items, and records UI reachability reasons.
Excel export	Bundled `ExcelJS` plus encoded workbook template	Creates validation workbook with summary and assessment-level sheets.
Word export	Custom OpenXML `.docx` generator	Creates Word review notes with formatted tables, sections, and clickable checkbox content controls.
Build output	`dist/` folder	Loadable production extension package.

Primary source folders:

- `api/`: API wrappers and request manager.
- `core/`: validation, review, assessment filtering, survey diff, DOCX generation.
- `checkpoints/`: Risk Profiler validation rules.
- `export/`: Excel export logic.
- `storage/`: Chrome local storage helpers.
- `utils/`: constants and shared helpers.

- `popup.html`, `popup.css`, `popup.js`: extension UI.
- `service_worker.js`: background job orchestration.
- `scripts/build.mjs`: build pipeline.

1(b). Permissions and What the Extension Does

Manifest permissions:

Permission	Why it is used
`storage`	Stores assessment lists, validation results, review results, failed assessment retry data, contexts, progress, and UI state.
`unlimitedStorage`	Prevents Chrome storage quota pressure when many assessments, contexts, and result sets are retained locally.
`alarms`	Schedules periodic assessment refresh every 30 minutes.
`tabs`	Finds/open trusted Cairo, ESATS, and GTC tabs and opens prerequisite links.
`scripting`	Executes fetch logic inside signed-in ESATS/GTC pages when direct extension-origin fetch is not sufficient.

Host permissions:

Host permission	Purpose
`https://cairois.web.boeing.com/*`	Cairo assessment, survey, template, answer, contact, and review-summary data.
`https://service-gateway.tas-phx.apps.boeing.com/*`	ESATS gateway application version and artifact data.
`https://termbank.web.boeing.com/*`	GTC vocabulary/API lookup for export-control artifacts.
`https://esats.web.boeing.com/*`	Trusted signed-in ESATS page used for token-backed gateway calls.
`https://gtc-ecm.web.boeing.com/*`	Trusted signed-in GTC page used for GTC calls.

What the extension does:

1. Checks that Cairo, ESATS, and GTC sessions are active.
2. Loads the primary Risk Profiler assessment list from Cairo.
3. Lets the user search/filter assessments by text, regex, status, owner, due date, or survey completed date.
4. Lets the user select assessments.

5. Runs validation mode against selected assessments using 13 checkpoint rules.
6. Runs review mode against selected assessments using old/new survey questions and answers. For incomplete assessments, reviewers can optionally run based on selected `newAnswers`.
7. Persists validation and review results separately in local browser storage.
8. Shows validation and review results in separate tabs.
9. Exports validation results to Excel.
10. Lets users choose review-note questions, enter ASA Notes, and download Word `.docx` review notes with clickable checkbox content controls.

1(c). Endpoints and HTTP Requests

All automation calls are read-only `GET` requests. The extension does not create, update, submit, or delete assessment records.

Cairo Endpoints

Endpoint	Method	Triggered by	Purpose
`https://cairois.web.boeing.com/api/asset/4/82/assessment/type/35`	`GET`	Assessment refresh, Cairo prerequisite check	Loads primary Risk Profiler assessment inventory.
`https://cairois.web.boeing.com/api/assessment/{id}/detail`	`GET`	Validation, review	Loads assessment detail, including survey template ID.
`https://cairois.web.boeing.com/api/assessment/survey/{id}/answers`	`GET`	Validation, review	Loads assessment survey answers.
`https://cairois.web.boeing.com/api/assessment/{id}/contacts`	`GET`	Legacy/available Cairo contact lookup	Loads assessment contacts if this wrapper is used by future review or validation logic. Current review notes use the ESATS contact-details summary endpoint.

Endpoint	Method	Triggered by	Purpose
`https://cairois.web.boeing.com/api/survey/template/{id}/questions`	`GET`	Validation, review, survey diff	Loads questions for a survey template.
`https://cairois.web.boeing.com/api/surveyTemplate/{id}`	`GET`	Review mode, survey diff	Loads survey-template metadata.
`https://cairois.web.boeing.com/api/surveyTemplate?where=alternateSurveyTemplateId:=:rp-app`	`GET`	Review mode, What's New modal	Loads all Risk Profiler app survey template versions.
`https://cairois.web.boeing.com/api/asset/4/{assetId}/assessment/review/summaries?assessmentTypeId=35&reviewTypeId=6`	`GET`	Validation mode	Loads review summary data used by checkpoint rules.
`https://cairois.web.boeing.com/api/assessment/survey/{assessmentId}/question/{surveyTemplateQuestionId}`	`GET`	Conditional validation in RP2/RP3	Loads question summary/collector data when URL evidence is needed.

ESATS Endpoints

Endpoint	Method	Triggered by	Purpose
`https://service-gateway.tas-phx.apps.boeing.com/gateway/asset/`	`GET`	Validation mode	Loads ESATS application versions.

Endpoint	Method	Triggered by	Purpose
`BusinessApplicationVersion/GetBusinessApplicationVersions?esatsId={assetId}`			
`https://service-gateway.tas-phx.apps.boeing.com/gateway/asset/BusinessApplicationVersionDocument/GetBusinessApplicationVersionPolicyAndArtifacts?esatsId={versionEsatsId}`	`GET`	Validation mode	Loads version policy/artifact data for each ESATS version.
`https://service-gateway.tas-phx.apps.boeing.com/gateway/asset/BusinessApplicationSummary/GetContactDetailsSummary?esatsId={assetId}`	`GET`	Review mode	Loads ESATS contact details used in review output and Word review notes.

ESATS gateway requests are executed from a trusted signed-in ESATS tab using `chrome.scripting.executeScript`. The extension reads the ESATS token from the ESATS page context and sends it as a bearer token when available.

GTC Endpoints

Endpoint	Method	Triggered by	Purpose
`https://termbank.web.boeing.com/ses/v1.2/GlobalTradeControlVocabularies/name/`	`GET`	Validation mode	Looks up GTC export-control vocabulary details for artifacts with `policyRuleId === 2`.

Endpoint	Method	Triggered by	Purpose
`{name}.json`			

GTC requests are also routed through a trusted signed-in GTC tab when required.

Session/Prerequisite Checks

URL	Method	Purpose
`https://cairois.web.boeing.com/api/asset/4/82/assessment/type/35`	`GET`	Verifies Cairo session/API access.
`https://service-gateway.tas-phx.apps.boeing.com/`	`GET`	Verifies ESATS gateway access.
`https://termbank.web.boeing.com/`	`GET`	Verifies GTC access.

2. Automation Statistics & Performance

Runtime Model

The extension uses browser-side concurrency:

- Validation concurrency: up to 5 assessments at a time.
- Review concurrency: up to 3 assessments at a time.
- Request retry policy: `fetchJson()` retries failed requests up to 3 attempts with a 1 second delay.
- Request cache: successful responses are cached in memory by URL during the current service-worker/runtime life, unless `useCache: false` is explicitly set.

Important distinction:

- Logical request count means how many unique API calls the automation intends to make.
- Network attempt count can be higher when retries are needed.
- Effective request count can be lower when cache hits occur.

2(a). Requests to Validate One Assessment

Validation builds a context and then runs RP1 through RP13 locally.

Per selected assessment, baseline validation requests:

Source	Count	Description
Cairo assessment detail	1	`assessment/{id}/`

Source	Count	Description
		`detail`
Cairo assessment answers	1	`assessment/survey/{id}/answers`
Cairo survey questions	1	`survey/template/{id}/questions`
Cairo review summary	1	`asset/4/{assetId}/assessment/review/summaries...`
ESATS versions	1	`GetBusinessApplicationVersions?esatsId={assetId}`
ESATS artifacts	`V`	One request per ESATS version.
GTC lookups	`E`	One request per export-control artifact where `policyRuleId === 2`.
Cairo question summary	`Q`	Conditional, currently up to 2 from RP2 and RP3 depending on answers.

Validation formula for one assessment:

```
Validation requests per assessment = 5 + V + E + Q
```

Where:

- `V` = number of ESATS versions for that application.
- `E` = number of export-control artifacts requiring GTC lookup.
- `Q` = conditional question-summary calls, normally 0 to 2.

Example:

If one assessment has 3 ESATS versions, 2 export-control artifacts, and 1 conditional URL summary call:

```
5 + 3 + 2 + 1 = 11 logical requests
```

Review Requests for One Assessment

Review mode loads the Risk Profiler survey template list once per run:

```
Global review run request = 1 request to surveyTemplate?
where=alternateSurveyTemplateId:=:rp-app
```

The extension can fall back to cached survey template data if the live template-list call fails.

Review Case 1: Incomplete Assessment Exists

For an assessment with an incomplete assessment ID in Initial Review Mode:

Request	Count
Last assessment detail	1
Last assessment survey questions	1
Last assessment answers	1
ESATS contact-details summary	1
Incomplete assessment detail	1
Incomplete assessment survey questions	1

Formula:

```
Incomplete initial review requests per assessment = 6
Run total = 1 + (6 * selected assessment count)
```

When Review Based on Selected Answers is enabled for incomplete assessments, the extension also loads incomplete assessment answers as `newAnswers`:

```
Incomplete selected-answer review requests per assessment = 7
Run total = 1 + (7 * selected assessment count)
```

Review Case 2: No Incomplete Assessment

For completed assessments with no incomplete assessment ID:

Request	Count
Last assessment detail	1
Last assessment survey questions	1
Last assessment answers	1
ESATS contact-details summary	1
Latest active survey template detail	1
Latest active survey template questions	1

Formula:

```
Completed review requests per assessment = 6
Run total = 1 + (6 * selected assessment count)
```

2(b). Requests for 10 Assessments

Validation for 10 Assessments

Formula:

```
10 * (5 + V + E + Q)
```

Examples:

Scenario	Formula	Logical requests
Minimum-like case: no ESATS versions, no GTC lookups, no conditional question summary	`10 * (5 + 0 + 0 + 0)`	50
Example case: 3 ESATS versions, 2 GTC artifacts, 1 conditional question summary per assessment	`10 * (5 + 3 + 2 + 1)`	110
Heavier case: 5 ESATS versions, 5 GTC artifacts, 2 conditional summaries per assessment	`10 * (5 + 5 + 5 + 2)`	170

Because validation runs 5 assessments concurrently, 10 assessments are processed in roughly 2 batches, subject to API response time and retries.

Review for 10 Assessments

Review scenario	Formula	Logical requests
10 completed assessments	`1 + (10 * 6)`	61
10 incomplete assessments in Initial Review Mode	`1 + (10 * 6)`	61
10 incomplete assessments in selected-answer review mode	`1 + (10 * 7)`	71
Mixed example: 6 completed + 4 incomplete in selected-answer review mode	`1 + (6 * 6) + (4 * 7)`	65

Because review runs 3 assessments concurrently, 10 assessments are processed in roughly 4 batches, subject to API response time and retries.

2(c). Inner Workings and Statistics Breakdown

Assessment Refresh

1. The service worker calls the Cairo primary assessment endpoint.
2. It normalizes the response into local assessment records:
    - `assetId`
    - `assetName`
    - `assessmentId`
    - `lastAssessmentId`
    - `incompleteAssessmentId`

    - `surveyCompletedOn`
    - `dueOn`
    - owner/manager fields
    - incomplete initiation fields
3. Records are stored in `chrome.storage.local`.
4. The refresh also runs automatically on extension install, browser startup, and every 30 minutes by Chrome alarm.

Validation Flow

1. User selects assessments.
2. Popup sends `START_VALIDATION` to the service worker.
3. Service worker creates a run ID and progress object.
4. `validateBatch()` processes up to 5 assessments at a time.
5. For each assessment, `buildContext()` gathers:
    - Cairo detail
    - Cairo answers, normalized by latest answer per `alternateQuestionId`
    - Cairo survey questions
    - Cairo review summary
    - ESATS versions
    - ESATS artifacts
    - GTC export-control vocabulary data
6. RP2 and RP3 can make conditional Cairo question-summary calls and extract URL evidence from either collector-style response data or nested JSON-style collected data.
7. `runValidation()` executes RP1 through RP13.
8. RP11, RP12, and RP13 treat unanswered service-account follow-up questions as `NA` when `CSIR-SvcAcct` is `Yes` and RP1 approvals are present.
9. RP11 passes explicit `Yes` or `No` values, fails other selected values, and only fails an unanswered question when the RP1 approval exception does not apply.
10. `scoreCalculator` computes pass/fail/N/A summary and score.
11. Results are stored under validation storage keys.
12. Popup renders validation cards and allows Excel export.

Review Flow

1. User selects assessments.

2. User can optionally open the review settings modal and choose a review mode. The default is Initial Review Mode.
3. Popup sends `START_REVIEW` to the service worker with the selected review mode.
4. Service worker creates a run ID and progress object.
5. `reviewBatch()` loads RP app survey template versions once.
6. Review runs up to 3 assessments at a time.
7. For each assessment:
    - It identifies `lastAssessmentId`.
    - It checks whether `incompleteAssessmentId` exists.
    - It loads old questions and answers from the last assessment.
    - It loads ESATS contact-details summary data and keeps `ApplicationManager` and `BusinessSystemManager`.
    - If incomplete: it loads the incomplete assessment questions and, when selected-answer mode is enabled, loads the incomplete assessment answers as `newAnswers`.
    - If completed: it ignores review mode settings, finds the latest released active RP app template, and loads its detail/questions.
8. The converted Python-to-JavaScript review algorithm:
    - uses `alternateQuestionId` as the stable question key,
    - selects the latest duplicate answer by `updatedOn`, then `createdOn`, then answer/template IDs,
    - maps old answers to new template questions,
    - validates carried answer values against new options,
    - evaluates route actions and branching logic,
    - avoids routing through conditions that depend on unreachable questions,
    - applies no-target route actions as branch guards so default survey order does not leak into hidden questions,
    - traverses reachable questions,
    - reports reachable unanswered work items with UI-only reachability reasons.
9. Results include review basis metadata: review mode, completed/incomplete behavior, old survey template ID, new survey template ID, and whether `newAnswers` was used.
10. Results are stored under review storage keys.
11. Popup renders review result cards.
12. User can open review notes, choose questions, add ASA Notes, and download Word review notes.

Review modes:

Mode	Applies to	Data used	Notes
Initial Review Mode	Completed and incomplete assessments	Old questions, old answers, and new questions	Default mode. Completed assessments always use this behavior.
Review Based on Selected Answers	Incomplete assessments only	Old questions, old answers, new questions, and `newAnswers`	Uses only answers currently present on the incomplete assessment. Completed assessments ignore this setting.

Word Review Notes Generation

The extension generates `.docx` files directly in the browser using OpenXML:

- Creates `[Content_Types].xml`
- Creates package relationships
- Creates `word/document.xml`
- Creates `word/styles.xml`
- Creates `word/settings.xml`
- Packages the files into a ZIP-compatible DOCX structure
- Downloads the file through a browser Blob

The Word output includes selected review questions, application metadata, ESATS contact details, `BEMS ID` where available, highlighted ASA Notes, and clickable checkbox content controls using WordprocessingML checkbox controls. The DOCX intentionally excludes UI-only Review Basis and reachability tooltip details. The downloaded file name is:

```
(app name)_RISK-PRofiler_review_notes.docx
```

3. Business & Technical Walkthrough

3(a). Business Management Walkthrough

What Problem This Solves

Before automation, reviewers must manually move between Cairo, ESATS, and GTC; inspect assessment details; compare survey versions; determine what changed or what remains unanswered; and manually create follow-up notes. This is repetitive and creates inconsistent output between reviewers.

This extension consolidates those tasks into one guided workflow.

Business Workflow

1. Open Chrome and sign in to Cairo, ESATS, and GTC.
2. Open the extension.
3. Confirm prerequisite sessions are active.
4. Search/filter the assessment list.
5. Select one or more applications.
6. Click `Validate Selected` to run automated quality checks.
7. Click `Review Selected` to generate review findings.
8. Use the settings icon next to `Review Selected` when an incomplete-assessment review should use selected `newAnswers`.
9. Review results in separate tabs.
10. Open review notes, select the questions to include, and enter ASA Notes when needed.
11. Export validation results to Excel when needed.
12. Download review notes to Word when business-ready documentation is needed.

Key Outputs

Validation output:

- Per-application score
- PASS/FAIL/N/A checkpoint details
- Excel workbook for reporting

Review output:

- Completed/incomplete assessment status
- Survey completed/due/incomplete initiated dates
- Review item count
- Review basis details in the UI, including review mode and survey template IDs
- UI-only reachability details explaining why each question appeared
- ApplicationManager and BusinessSystemManager contact details
- Word notes with selected checklist items, ASA Notes, and clickable checkbox controls

Impact

The extension improves:

- Review consistency
- Turnaround time
- Repeatability

- Evidence gathering
- Management visibility
- Handoff quality between review teams and application owners

What the Business Should Know

- The extension does not submit or modify assessments.
- It reads existing assessment, template, contact, ESATS, and GTC data.
- It stores results locally in Chrome.
- It stores selected review mode and ASA Notes locally in Chrome.
- It creates review and validation artifacts that can be shared with stakeholders.
- Review Basis and reachability notes help users understand review output in the UI, but are not written into downloaded Word review notes.
- Final business decisions still belong to the reviewer or assessment owner.

3(b). Technical Project Manager Walkthrough

Architecture

The application follows a modular browser-extension architecture:

- Popup UI handles user interaction.
- Service worker handles long-running automation tasks.
- API modules centralize endpoint access.
- Core modules implement validation, review, filtering, and document generation.
- Storage helpers isolate Chrome local storage.
- Build script bundles deployable assets into `dist/`.

Data Flow

```
User action
  -> popup.js
  -> chrome.runtime message
  -> service_worker.js
  -> API modules
  -> core validation/review logic
  -> chrome.storage.local
  -> popup render
  -> Excel/DOCX export if requested
```

Validation Components

- `core/contextBuilder.js`: collects all context.
- `core/batchValidator.js`: concurrency and progress.
- `core/validationEngine.js`: runs checkpoints.

- `core/checkpointRegistry.js`: registers RP1-RP13.
- `checkpoints/*.js`: individual validation logic.
- `export/excelExporter.js`: Excel workbook output.

Review Components

- `core/reviewEngine.js`: review data gathering and reachable unanswered work queue algorithm.
- `core/reviewNotesDocx.js`: Word document generation.
- `core/answerUtils.js`: answer-list loading and latest-answer normalization by `alternateQuestionId`.
- `storage/storage.js`: review result persistence.
- `popup.js`: review UI and download handlers.

State Management

Stored keys include:

- `assessments`
- `validations`
- `reviews`
- `validationProgress`
- `reviewProgress`
- `validationComplete`
- `reviewComplete`
- `failedAssessments`
- `assessmentContexts`
- `lastAction`
- `whatsNewModalState`
- `reviewMode`
- `reviewQuestionNotes`

Popup runtime state also keeps the active review-note question selections so the modal and DOCX download use the same selected question set.

Error Handling

- API calls retry up to 3 times.
- Validation failures are stored per assessment where possible.
- Review failures are stored per assessment where possible.

- Failed validations can be retried.
- Review template-list failure can fall back to cached template versions.
- Progress UI shows completed count, elapsed time, estimated remaining time, and final processing time.

Security Notes

- The extension relies on existing authenticated browser sessions.
- Cairo calls use cookies with `credentials: include`.
- ESATS and GTC calls may run inside trusted signed-in tabs through `chrome.scripting`.
- The extension does not ask users for passwords.
- The extension does not write back to Cairo, ESATS, or GTC.
- Output artifacts are generated locally in the browser.

4. Enterprise Production Readiness

To scale this extension for enterprise-level production use, the following controls and enhancements are recommended.

Deployment and Governance

- Publish through an enterprise Chrome Web Store or managed extension deployment.
- Lock extension ID and versioning.
- Use controlled release channels: development, pilot, production.
- Require formal change approvals for endpoint, permission, and validation-rule changes.
- Maintain a release note for each version.

Security Hardening

- Review and approve all host permissions.
- Keep host permissions limited to required Boeing domains.
- Add content security policy review during release.
- Avoid broad `<all_urls>` web-accessible resources unless explicitly justified.
- Consider moving any sensitive endpoint configuration into a centrally managed configuration file.
- Perform a security review of `chrome.scripting` usage because it executes code in trusted pages.
- Add static analysis and dependency scanning to the build pipeline.

Performance and Scale

- Keep validation concurrency configurable.
- Keep review concurrency configurable.
- Add API rate-limit backoff if upstream systems require it.
- Add request telemetry counters for:
    - successful requests,
    - failed requests,
    - retries,
    - cache hits,
    - average processing time per assessment.
- Add a maximum batch size guardrail for very large selections.
- Consider queueing long-running jobs with pause/resume semantics.

Observability

- Add a diagnostics panel for:
    - current extension version,
    - last refresh time,
    - assessment count,
    - last validation/review run ID,
    - request statistics,
    - failed endpoint summary.
- Add exportable diagnostic logs that do not include sensitive tokens.
- Add user-friendly error messages mapped to common authentication/session failures.

Data Handling

- Define retention policy for local validation/review results.
- Add a "clear all local data" administrative option.
- Consider encrypting sensitive locally stored context if policy requires it.
- Avoid storing unnecessary raw API payloads long term.
- Review generated Excel and Word artifacts for classification/handling requirements.

Reliability

- Add automated unit tests for:
    - assessment filtering,
    - review traversal,

    - answer normalization,
    - template version selection,
    - DOCX XML generation,
    - checkpoint scoring.
- Add mocked API integration tests for validation and review workflows.
- Add regression fixtures for survey template changes and answer carry-forward cases.
- Add a build validation step that loads the extension package in a headless browser.

Maintainability

- Keep endpoints centralized in `utils/constants.js`.
- Keep each checkpoint isolated in `checkpoints/`.
- Document each checkpoint's business rule, input questions, and expected output.
- Version the review algorithm when business logic changes.
- Maintain sample payloads for primary assessments, survey templates, answers, contacts, and review output.

Enterprise Operating Model

Recommended production operating model:

1. Product owner owns business rules and checkpoint acceptance.
2. Technical owner owns extension architecture and release integrity.
3. Security owner reviews permissions and data handling.
4. Pilot group validates output against real assessments.
5. Production support monitors failures and gathers enhancement requests.
6. Release cadence follows formal enterprise change windows.

Future Enhancements

- Centralized configuration for concurrency and endpoint toggles.
- Server-side telemetry dashboard for aggregate performance without storing sensitive assessment details.
- Role-based feature flags.
- Bulk Word review-notes download as a ZIP.
- Automated comparison report between survey template versions.
- More detailed endpoint/request summary after each run.
- In-product help text for each checkpoint and review tag.

- Optional integration with enterprise ticketing/workflow systems.

Build and Load Instructions

Install dependencies:

```
npm install
```

Build the extension:

```
npm run build
```

Load in Chrome:

1. Open `chrome://extensions`.
2. Enable Developer mode.
3. Click `Load unpacked`.
4. Select the `dist/` folder.
5. Open/sign in to Cairo, ESATS, and GTC before running validation or review.

Current Version

```
Extension version: 1.0.0
Manifest: Chrome Extension Manifest V3
Primary modes: Validation Mode, Initial Review Mode, and Review Based on
Selected Answers
```

Developer Guide: Modifying, Adding, or Removing Checkpoints

This section explains how developers maintain the validation checkpoint system. Checkpoints are the business rules executed when a user clicks `Validate Selected`.

Checkpoint Architecture

Checkpoint files live in:

```
checkpoints/
```

The registry that controls which checkpoints run, and in what order, is:

```
core/checkpointRegistry.js
```

The validation engine that executes the registered checkpoints is:

```
core/validationEngine.js
```

The shared helper functions used by checkpoint files are:

```
checkpoints/helpers.js
```

Current checkpoints:

```
RP1.js
RP2.js
RP3.js
RP4.js
RP5.js
RP6.js
RP7.js
RP8.js
RP9.js
RP10.js
RP11.js
RP12.js
RP13.js
```

Each checkpoint exports one object with this shape:

```
const RP99 = {
    id: "RP99",
    name: "Human-readable checkpoint name",
    category: "Checkpoint category",
    requiredQuestions: [
        "Question-Alternate-ID"
    ],

    async validate(context) {
        // checkpoint logic
        return pass(this.id, "Reason shown in the UI and Excel export.");
    }
};

export default RP99;
```

Context Available to Checkpoints

Every checkpoint receives a `context` object built by `core/contextBuilder.js`.

Important context fields:

Field	Description
`context.application`	Normalized assessment list item from Cairo primary assessment data.
`context.assessment`	Cairo assessment detail response.
`context.answers`	Cairo assessment survey answers. Helper functions normalize duplicate `alternateQuestionId` records to the latest answer before returning values.
`context.surveyQuestions`	Cairo survey template questions.
`context.questionMap`	Map keyed by `alternateQuestionId`.
`context.reviewSummary`	Cairo review summary data for approval/review-related checks.
`context.versions`	ESATS business application versions.
`context.artifacts`	ESATS policy/artifact data.
`context.exportControl`	GTC lookup responses for export-control artifacts.

Standard Checkpoint Return Values

Use helper functions from `checkpoints/helpers.js`:

```
import {
    pass,
    fail,
    notApplicable
} from "./helpers.js";
```

Return values:

Helper	Status	Meaning
`pass(id, reason)`	`PASS`	Checkpoint passed.
`fail(id, reason)`	`FAIL`	Checkpoint failed and needs attention.
`notApplicable(id, reason)`	`NA`	Checkpoint does not apply to this assessment.

The `reason` text is important. It is shown in:

- Validation Results UI
- Downloaded context/details
- Excel export
- Summary/error reporting

Required Questions

If a checkpoint depends on one or more survey questions, add `requiredQuestions`.

Example:

```
requiredQuestions: [
    "CSIR-AppType",
    "CSIR-MFA"
]
```

Before a checkpoint runs, `core/validationEngine.js` checks whether each required question exists in `context.questionMap`. If a required question is missing, the checkpoint is marked `NA` with the reason:

```
Question identifier was not found in the survey questions.
```

Use `requiredQuestions` when the checkpoint cannot produce a meaningful result without a specific Risk Profiler question.

Common Helper Functions

Useful helpers in `checkpoints/helpers.js`:

Helper	Purpose
`getAnswer(context, questionId)`	Gets one answer by `alternateQuestionId`.
`getAnswers(context)`	Returns a normalized answer list. Duplicate `alternateQuestionId` records are reduced to the latest answer by `updatedOn`, then `createdOn`, then ID tie breakers.
`getValues(context, questionId)`	Extracts answer values for a question.
`hasAnswer(context, questionId)`	Checks whether a question has any answer value.
`includesValue(context, questionId, expected)`	Checks exact normalized answer match.
`includesAnyValue(context, questionId, expectedValues)`	Checks whether answer matches any expected values.
`valueContainsAny(context, questionId, fragments)`	Checks partial normalized matches.
`isYes(context, questionId)`	Checks whether answer is `Yes`.
`isNo(context, questionId)`	Checks whether answer is `No`.
`isSaas(context)`	Checks SaaS-style app type.
`isWebLikeApplication(context)`	Checks web/API/SaaS/PaaS style app type.
`hasRiskProfilerApprovals(context)`	Checks whether Risk Profiler approval/review summary data supports approval-dependent checkpoint exceptions.
`extractHttpUrls(node)`	Walks a nested payload and returns unique HTTP/HTTPS URLs from collector-style or JSON-style question-summary responses.
`findValuesByKeyFragment(node, fragments)`	Searches nested objects for key fragments.
`collectValuesByKey(node, keyName)`	Collects nested values by exact key.
`findAssessmentById(node, assessmentId)`	Finds assessment object in nested review summary.
`deploymentPhase(context)`	Gets lifecycle/deployment phase from assessment/version context.
`getQuestionSummary(context, surveyTemplateQuestionId)`	Makes an extra Cairo question-summary API call. Use only when needed.

How to Modify an Existing Checkpoint

1. Open the checkpoint file in `checkpoints/`.

   Example:

```
checkpoints/RP5.js
```

2. Review the existing checkpoint object:
    - `id`
    - `name`
    - `category`
    - `requiredQuestions`
    - `validate(context)`
3. Change only the business logic needed inside `validate(context)`.
4. If the rule now depends on different survey questions, update `requiredQuestions`.
5. Keep the `id` stable unless the checkpoint is being renamed by business decision. Excel export and reporting rely on checkpoint IDs.
6. Return `pass`, `fail`, or `notApplicable` with clear reason text.
7. Run:

```
npm run build
```

8. Load the `dist/` extension and test the checkpoint against:
    - a passing assessment,
    - a failing assessment,
    - an assessment where the rule is not applicable,
    - an assessment where required questions may be missing.

How to Add a New Checkpoint

1. Choose the next checkpoint ID.

   Example:

```
RP14
```

2. Create a new file:

```
checkpoints/RP14.js
```

3. Use this starter template:

```
import {
    fail,
    getValues,
    notApplicable,
    pass
} from "./helpers.js";

const RP14 = {
    id: "RP14",
    name: "Describe the checkpoint in business language",
    category: "Architecture Overview",
    requiredQuestions: [
```

```
        "Question-Alternate-ID"
    ],

    async validate(context) {
        const values =
            getValues(
                context,
                "Question-Alternate-ID"
            );

        if (
            values.length === 0
        ) {
            return notApplicable(
                this.id,
                "Required answer was not provided."
            );
        }

        const passed =
            values.includes("Expected Value");

        return passed
            ? pass(
                this.id,
                "Expected value was selected."
            )
            : fail(
                this.id,
                "Expected value was not selected."
            );
    }
};

export default RP14;
```

4. Register the checkpoint in `core/checkpointRegistry.js`.

   Add an import:

```
import RP14 from "../checkpoints/RP14.js";
```

   Add it to the `CHECKPOINTS` array in the intended execution order:

```
export const CHECKPOINTS = [
    RP1,
    RP2,
    RP3,
    // ...
    RP13,
    RP14
];
```

5. If the Excel template has a dedicated row for checkpoint IDs, update the template so the new `RP14` row exists. The exporter discovers rows by matching checkpoint IDs such as `RP1`, `RP2`, etc. If the workbook template does not contain `RP14`, the checkpoint can still appear in summary data, but it will not populate a dedicated detailed row in the cloned assessment sheet.
6. If the new checkpoint requires additional API data not currently in `context`, update

`core/contextBuilder.js` carefully.

Before adding new API calls, confirm:

- which endpoint is needed,
- whether the request can be shared across checkpoints,
- whether it increases per-assessment request volume,
- whether it requires new host permissions,
- whether it should be cached.

7. If a new endpoint is needed, add it to:

```
utils/constants.js
```

Then add an API wrapper in the appropriate module under:

```
api/
```

8. Run:

```
npm run build
```

9. Test the new checkpoint in the UI and Excel export.

How to Remove a Checkpoint

1. Open:

```
core/checkpointRegistry.js
```

2. Remove the checkpoint import.

Example:

```
import RP14 from "../checkpoints/RP14.js";
```

3. Remove the checkpoint from the `CHECKPOINTS` array.
4. Decide whether to keep or delete the file under `checkpoints/`.

Recommended approach:

- Keep the file temporarily if removal is experimental.
- Delete the file when the checkpoint is formally retired.

5. If the Excel template has a dedicated row for the removed checkpoint, decide whether to remove it from the template or leave it blank for historical consistency.
6. Update documentation and release notes so business users know the checkpoint is no longer part of scoring.
7. Run:

```
npm run build
```

8. Validate that:
    - the removed checkpoint no longer appears in Validation Results,
    - the score calculation still behaves as expected,
    - Excel export does not show unexpected blank/error data.

How to Rename or Reorder a Checkpoint

Rename display text:

- Update `name` in the checkpoint file.
- Keep `id` unchanged unless reporting requirements explicitly change.

Change category:

- Update `category` in the checkpoint file.

Change execution/display order:

- Reorder entries in `CHECKPOINTS` inside `core/checkpointRegistry.js`.

Important: changing an `id` can affect Excel row mapping, historical comparisons, and stakeholder references. Prefer changing `name` instead of `id` when only wording changes.

When a Checkpoint Needs Extra API Data

Most checkpoint logic should use existing `context`. If a checkpoint needs new data:

1. Add endpoint constant in `utils/constants.js`.
2. Add an API function in `api/`.
3. Add the API call in `core/contextBuilder.js` if multiple checkpoints can reuse it.
4. If the API call is only needed conditionally, consider calling it inside the checkpoint instead, similar to RP2/RP3 using `getQuestionSummary`.
5. Document the new request in README endpoint and performance sections.
6. Recalculate request-count formulas if the request runs for every assessment.

Use conditional calls for expensive or rarely needed lookups. Use context-level calls for data required by many checkpoints.

For question-summary payloads that may vary between collector-style structures and JSON-style collected-data structures, prefer `extractHttpUrls()` or shared traversal helpers instead of hardcoding one response shape.

Checkpoint Testing Checklist

Before promoting a checkpoint change:

- Confirm the extension builds with `npm run build`.
- Confirm the checkpoint appears in Validation Results.

- Confirm PASS, FAIL, and N/A paths all return clear reason text.
- Confirm missing required questions are handled correctly.
- Confirm no unnecessary endpoint calls were added.
- Confirm Excel export still works.
- Confirm the scoring summary still makes business sense.
- Confirm the rule wording is understandable by business users.
- Confirm any new endpoint is documented in this README.

Code Quality Expectations

Checkpoint code should be:

- deterministic,
- readable,
- scoped to one business rule,
- explicit about why it passes or fails,
- defensive against missing/null API fields,
- conservative with new network requests,
- consistent with existing helper usage.

Avoid:

- hardcoding assessment-specific values,
- making write/update API calls,
- adding broad host permissions for one checkpoint,
- burying business rules in generic helpers,
- returning vague reasons such as `failed validation`.

Minimal Files Changed by Checkpoint Work

Common modification:

```
checkpoints/RP*.js
```

Adding a checkpoint:

```
checkpoints/RP14.js
core/checkpointRegistry.js
README.md
```

Adding a checkpoint with new API data:

```
checkpoints/RP14.js
core/checkpointRegistry.js
utils/constants.js
api/<appropriateApiModule>.js
```

```
core/contextBuilder.js
README.md
```

Removing a checkpoint:

```
core/checkpointRegistry.js
checkpoints/RP*.js
README.md
```